

Лабораторная работа №5

Основы языка Ассемблера

Цели работы: изучить теоретические основы языка Ассемблера, научиться писать программы, реализующие ввод, вывод и обработку числовой информации с использованием команд языка Ассемблера.

Теоретическая часть

Ассемблер (от англ. assembler — сборщик) — компьютерная программа, компилятор исходного текста программы, написанной на языке Ассемблера, в программу на машинном языке.

Как и сам язык (Ассемблер), ассемблеры, как правило, специфичны конкретной архитектуре, операционной системе и варианту синтаксиса языка. Вместе с тем существуют мультиплатформенные или вовсе универсальные (точнее, ограниченно-универсальные, потому что на языке низкого уровня нельзя написать аппаратно-независимые программы) ассемблеры, которые могут работать на разных платформах и операционных системах. Среди последних можно также выделить группу кросс-ассемблеров, способных собирать машинный код и исполняемые модули (файлы) для других архитектур и ОС.

Общие положения о программировании в Windows, используемые в языке Ассемблера для ОС Windows:

- Программирование в Windows основывается на использовании функций API (Application Program Interface, т.е. интерфейс программного приложения). Их количество достигает двух тысяч. Программа на ассемблере в значительной степени будет состоять из таких вызовов. Все взаимодействие с внешними устройствами и ресурсами операционной системы будет происходить посредством таких функций.
- Главным элементом программы в среде Windows является окно. Для каждого окна определяется своя процедура обработки сообщений.
- Окно может содержать элементы управления: кнопки, списки, окна редактирования и др. Эти элементы, по сути, также являются окнами, но обладающими особыми свойствами. События, происходящие с этими элементами (и самим окном), приводят к приходу сообщений в процедуру окна.
- Операционная система Windows использует линейную модель памяти, соответственно всю память можно рассматривать как один сегмент. Для программиста на языке Ассемблера это означает, что адрес любой ячейки памяти будет определяться содержимым одного 32-битного регистра.

Регистры микропроцессора

В программах на языке Ассемблера одними из основных элементов являются регистры. Регистр - это определенный участок памяти внутри самого процессора, от 8-ми до 32-х бит длиной, который используется для

промежуточного хранения информации, обрабатываемой процессором. Некоторые регистры содержат только определенную информацию.

Регистры общего назначения - EAX, EBX, ECX, EDX. Они 32-битные и делятся еще на две части, нижние из которых AX, BX, CD, DX – 16-битные, и делятся еще на два 8-битных регистра. Так, AX делится на AH и AL, DX на DH и DL и т.д. Буква "H" означает верхний регистр.

Так, AH и AL каждый по одному байту, AX - 2 байта (или word - слово), EAX - 4 байта (или dword - двойное слово). Эти регистры используются для операций с данными, такими как сравнение, математические операции или запись данных в память.

Регистры сегментов - это CS, DS, ES, FS, GS, SS. Эти регистры 32-битные, и содержат в себе первую половину адреса, используемую микропроцессором при адресации.

CS - сегмент кода (страница памяти) исполняемой в данный момент программы.

DS - сегмент (страница) данных исполняемой программы, т.е. константы, строковые ссылки и т.д.

SS - сегмент стека исполняемой программы.

ES, FS, GS - дополнительные сегменты, могут не использоваться программой.

Регистры смещения - EIP, ESP, EBP, ESI, EDI. Эти регистры 32-битные, нижняя половина которых доступна как регистры IP, SP, BP, SI, DI.

EIP - указатель команд, содержит величину смещения относительно начала программы, которая будет исполняться следующей. То есть полный адрес на следующую исполняемую линию кода будет ESI:EIP.

Регистр ESP указывает на адрес вершины стека (адрес, куда будет заноситься следующая переменная командой PUSH).

Регистр EBP содержит адрес, начиная с которого в стек вносится или забирается информация (или "глубина" стека). Параметры функций имеют положительный сдвиг относительно EBP, локальные переменные - отрицательный сдвиг, а полный адрес этого участка памяти будет SS:EBP.

Регистр ESI - адрес источника, содержит адрес начала блока информации для операции "переместить блок" (полный адрес DS:SI), а регистр EDI- адрес назначения в этой операции (полный адрес ES:EDI).

Операнды языка Ассемблера

Данные, которые обрабатываются командами, называются операндами. Операнды в языке Ассемблера записываются непосредственно после команды; если их несколько, то через запятую. Одни команды вообще не имеют никаких операндов, другие имеют один или два операнда.

В качестве операнда можно указать непосредственное значение (например, 123), имя регистра или ссылку на ячейку памяти (так называемые косвенные операнды).

Что же касается разрядности, имеются 32-разрядные, 16-разрядные и 8-разрядные операнды. Почти каждая команда требует, чтобы операнды были одинакового размера (разрядности).

Адресация памяти

Чтобы не запоминать адреса всех «переменных», используемых в программе, этим адресам присваивают символические обозначения, которые называются переменными (иногда их также называют указателями). При использовании косвенного операнда адрес в памяти, по которому находится нужное значение, записывается в квадратных скобках: [адрес]. Если мы используем указатель, т.е. символическое представление адреса, например [ESI], то в листинге машинного кода мы увидим, что указатель был заменен реальным значением адреса. Можно также указать точный адрес памяти, например [594FH].

Чаще всего мы будем адресовать память по значению адреса, занесенному в регистр процессора. Чтобы записать такой косвенный операнд, нужно просто написать имя регистра в квадратных скобках. Например, если адрес загружен в регистр ESI, вы можете получить данные, расположенные по этому адресу, используя выражение [ESI].

Имеются даже более сложные типы адресации: [адрес + EBX + 4]. В этом случае процессор складывает адрес, значение 4 и значение, содержащееся в регистре EBX. Результат этого выражения называется эффективным адресом (EA, Effective Address) и используется в качестве адреса, по которому фактически находится операнд.

В адресации памяти участвуют сегментные регистры. Их функция зависит от режима процессора. Каждый способ адресации предполагает, что при вычислении реального (фактического) адреса используется сегментный регистр по умолчанию.

Команды языка Ассемблера

Общий формат записи команд языка Ассемблера такой:

имя_команды [подсказка] операнды

Необязательная подсказка указывает компилятору требуемый размер операнда. Ее значением может быть слово BYTE (8-битный операнд), WORD (16-битный) или DWORD (32-битный).

Рассмотрим основные команды, используемые в данной лабораторной работе.

Команда MOV копирует значение из источника в приемник:

Mov ax,[number]	заносим значение переменной number в регистр AX
Mov [number],bx	загрузить значение регистра BX в переменную number
Mov bx, cx	занести в регистр BX значение регистра CX
Mov al, 1	занести в регистр AL значение 1
Mov dh, cl	занести в регистр DH значение регистра CL
Mov esi,edi	копировать значение регистра EDI в регистр ESI
Mov word [number],1	сохранить 16-битное значение 1

Команда сложения **ADD** требует двух операндов, как и команда **MOV**:
ADD ol, o2.

Команда **ADD** складывает оба операнда и записывает результат в **o1**, предыдущее значение которого теряется. Точно так же работает команда вычитания — **SUB**:

SUB ol, o2.

Результат **o1-o2** будет сохранен в **o1**, исходное значение **o1** будет потеряно. Рассмотрим пример:

```
mov  eax, 8  ;вносим в EAX число 8
mov  ecx, 6  ;вносим в ECX число 6
mov  edx, cx ;копируем ECX в EDX, EDX = 6
add  edx, eax;EDX = EDX + EAX.
```

Поскольку мы хотим избежать уничтожения (т.е. перезаписи результатов) исходных значений и хотим сохранить оба значения — **EAX** и **ECX**, мы копируем оригинальное значение **ECX** в **EDX**, а затем добавляем **EAX** к **EDX**. Команда **ADD** сохранит результат **EDX + EAX** в регистре **EDX**, а исходные значения **EAX** и **ECX** останутся неизменными.

Команда **INC** добавляет, а **DEC** вычитает единицу из единственного операнда. Допустимые типы операнда — такие же, как у команд **ADD** и **SUB**, а формат команд таков:

```
INC  ol      ;ol = ol + 1
DEC  ol      ;ol = ol - 1.
```

Операции умножения и деления в языке Ассемблера имеют свою специфику. В результате умножения двух чисел мы можем получить число, диапазон которого будет в два раза превышать диапазон операндов. Деление целых чисел — это операция целочисленная, поэтому в результате образуются два значения: частное и остаток.

С целью упрощения реализации команд умножения и деления эти команды спроектированы так, что один из операндов и результат находятся в фиксированном регистре, а второй операнд указывается программистом.

Подобно командам **ADD** и **SUB**, команды **MUL**, **DIV**, **IMUL**, **IDIV** изменяют регистр признаков.

Команда **MUL** может быть записана в трех различных форматах — в зависимости от операнда:

В 8-разрядной форме операнд может быть любым 8-битным регистром или адресом памяти. Второй операнд всегда хранится в **AL**. Результат (произведение) будет записан в регистр **AX**.

В 16-разрядной форме операнд может быть любым 16-битным регистром или адресом памяти. Второй операнд всегда хранится в **AX**. Результат сохраняется в паре **DX:AX**.

(Регистр/память16) * **AX** -> **DX:AX**

В 32-разрядной форме второй операнд находится в регистре **EAX**, а результат записывается в пару **EDX:EAX**.

(Регистр/память32) * **EAX** -> **EDX:EAX**

Рассмотрим несколько примеров.

Пример 1: умножить значения, сохраненные в регистрах BH и CL, результат сохранить в регистр AX:

mov al, bh ;AL = BH — сначала заносим в AL второй операнд

*mul cl ;AX = AL * CL — умножаем его на CL.*

Результат будет сохранен в регистре AX.

Пример 2: вычислить 48^2 , результат сохранить в DX:AX:

mov ax, 486 ; AX = 486

*mul ax ; AX * AX -> DX:AX.*

Команда **IMUL** умножает целые числа со знаком и может использовать один, два или три операнда. Когда указан один операнд, то поведение IMUL будет таким же, как и команды MUL, просто она будет работать с числами со знаком.

Если указано два операнда, то инструкция IMUL умножит первый операнд на второй и сохранит результат в первом операнде, поэтому первый операнд всегда должен быть регистром. Вторым операндом может быть регистром, непосредственным значением или адресом памяти.

*imul ecx, 6 ;ECX = ECX * 6.*

Если указано три операнда, то команда IMUL перемножит второй и третий операнды, а результат сохранит в первый операнд. Первый операнд — только регистр, второй может быть любого типа, а третий должен быть только непосредственным значением:

*imul edx, ecx, 7 ;EDX = ECX * 7*

*imul ebx, [sthing], 9 ;EBX=знач.переменной «sthing» * 9*

*imul ecx, edx, 11 ;ECX = EDX * 11.*

Подобно команде MUL, команда **DIV** может быть представлена в трех различных форматах в зависимости от типа операнда.

Операнд служит делителем, а делимое находится в фиксированном месте. В 8-битной форме переменный операнд (делитель) может быть любым 8-битным регистром или адресом памяти. Делимое содержится в AX. Результат сохраняется так: частное в AL, остаток в AH.

В 16-битной форме операнд может быть любым 16-битным регистром или адресом памяти. Вторым операндом всегда находится в паре DX:AX. Результат сохраняется в паре DX:AX (DX — остаток, AX — частное).

В 32-разрядной форме делимое находится в паре EDX:EAX, а результат записывается в пару EDX:EAX (частное в EAX, остаток в EDX).

Команда **IDIV** используется для деления чисел со знаком, синтаксис ее такой же, как у команды DIV.

Рассмотрим несколько примеров:

Пример 1: разделить 13 на 2, частное сохранить в BL, а остаток в — ВН:

```
mov ax,13    ;AX = 13
mov cl,2     ; CL = 2
div cl       ;делим на CL
mov bx,ax    ;ожидаемый результат находится в AX.
```

Пример 2: вычислить радиус по диаметру, значение которого сохранено в 16-битной переменной diameter, результат записать в radius, а остаток проигнорировать.

```
mov ax,[diameter] ;AX = diameterl
mov cl,2          ;загружаем делитель 2
div cl           ;делим
mov [radius],al   ;сохраняем результат.
```

Постановка задачи

1. Изучить теоретические основы языка Ассемблера.
2. Запустить программу fasmw.exe.
3. Открыть в программе fasmw.exe файл lab5.asm.
4. Внимательно изучить текст программы lab5.asm по имеющимся в ней комментариям.
5. Запустить программу на выполнение (клавиша F9).
6. Модифицировать текст программы в соответствии с вариантом задания.

Задания для самостоятельного выполнения

Варианты заданий.

1) $y=a+b-c*d/a$	2) $y=a+b-c/d*a$	3) $y=a+b*c-d/a$
4) $y=a+b*c*d-d$	5) $y=a+b/c-d*a$	6) $y=a+b/c*d-d$
7) $y=a-b+c*d/a$	8) $y=a-b+c/d*a$	9) $y=a+b-c*d/a$
10) $y=a-b*c/d+c$	11) $y=a-b/c+d*a$	12) $y=a-b/c*d+a$
13) $y=a+b-c*d/a$	14) $y=a+b*c/d-c$	15) $y=a-b+c/d*d$
16) $y=a*b+c-d/a$	17) $y=a*b+c/d-a$	18) $y=a*b-c+d/a$
19) $y=a*b-c/d+a$	20) $y=a*b/c+d-a$	21) $y=a*b/c-d*a$
22) $y=a/b+c-d*a$	23) $y=a/b+c*d-a$	24) $y=a/b-c+d*a$
25) $y=a/b-c*d+a$	26) $y=a/b*c+d-a$	27) $y=a/b*c-d+a$
28) $y=a*b+c*d/a$	29) $y=a*b-c/d+a$	30) $y=a/b+c*d-a$

